



HAUTE
ÉCOLE
ALBERT
JACQUARD

HEHbe
Haute École en Hainaut



HAUTE ÉCOLE
CONDORCET

UMONS
Université de Mons

UCLouvain

Département des industries créatives et numériques

Simulation physique de cordes dans un jeu vidéo

Dans le cadre d'un jeu vidéo, comment simuler de manière crédible et fluide des cordes dans un environnement 3D dynamique ?

Mémoire présenté par

CAYÈRÉ, Clément

*En vue de l'obtention du diplôme de
Master en Jeu vidéo*

Sous la supervision de

CHAPELLE, Joakim, superviseur

DAMOISEAUX, Gil, coach

Année académique 2024-2025

Résumé

Dans ce mémoire, nous allons chercher à simuler le comportement d'une corde dans le moteur de jeu Unreal Engine 5. Cette simulation a pour finalité d'être intégrée dans un jeu vidéo qui utilise des cordes comme mécanique principale. Nous verrons quelles méthodes ont été utilisées pour simuler cette corde de manière plausible, que ce soit au niveau de son comportement ou de ses interactions avec un environnement dynamique.

Mots-clefs

Jeu vidéo, Corde, Simulation physique, Position Based Dynamic, Collisions, Contraintes.

TABLE DES MATIERES

1	INTRODUCTION.....	6
2	PROBLEMATIQUE.....	7
3	ÉTAT DE L'ART.....	8
3.1	RAPPELS ET NOTIONS ELEMENTAIRES	8
3.1.1	<i>Simulation d'une corde</i>	8
3.1.2	<i>Intégration</i>	8
3.1.2.1	Méthode d'Euler.....	9
3.1.2.2	Méthode de Verlet	9
3.2	CONFRONTATION DES APPROCHES DE SIMULATION	10
3.2.1	<i>Approche basée sur des forces</i>	10
3.2.2	<i>Approche basée sur des vitesses</i>	11
3.2.3	<i>Approche basée sur les positions</i>	11
3.2.3.1	Première version	11
3.2.3.2	PBD	12
3.3	CABLE COMPONENT D'UNREAL ENGINE	13
3.4	AMELIORATIONS DU SYSTEME	14
3.4.1	<i>Small steps</i>	14
4	METHODOLOGIE	15
4.1	PREMIER PROTOTYPE.....	15
4.2	POC DE LA CORDE.....	15
4.2.1	<i>Conception</i>	16
4.2.2	<i>Algorithme d'actualisation</i>	16
4.2.3	<i>Contrainte de distance</i>	17
4.2.4	<i>Collision et friction</i>	18
4.2.5	<i>Interface de design</i>	18
4.3	INTEGRATION DE LA SOLUTION DANS LE TFE	19
4.3.1	<i>Actions du joueur sur la corde</i>	19
4.3.2	<i>Nouveau système de rendu</i>	19
4.3.3	<i>Système de visée automatique</i>	19
4.4	PLAYTESTS ET AJUSTEMENTS	19
4.5	OPTIMISATION	20
4.5.1	<i>Rope Manager</i>	20
4.5.2	<i>Substepping</i>	20
4.5.3	<i>Broad phase</i>	20
5	PRESENTATION DU PROTOTYPE.....	21
5.1	COMPORTEMENT ET SIMULATION	21
5.1.1	<i>Rendu et gravité</i>	21
5.1.2	<i>Tension</i>	22
5.1.3	<i>Collision et friction</i>	22
5.2	ACTIONS DU JOUEUR	23
5.2.1	<i>Actions de base</i>	23
5.2.2	<i>Manipulation des objets reliés</i>	24
5.2.3	<i>QoL Improvements</i>	25
5.2.3.1	Système de visée	25
5.2.3.2	Système de téléportation d'objets reliés.....	25
5.2.3.3	Facilitation des actions de base	25

5.3	FONCTIONNEMENT INTERNE	26
5.3.1	<i>Interface de personnalisation</i>	26
5.3.2	<i>Debug</i>	26
5.4	OPTIMISATION	26
6	ANALYSE REFLEXIVE	28
6.1	DISCUSSION.....	28
6.1.1	<i>Choix créatifs</i>	28
6.1.2	<i>Choix techniques</i>	28
6.2	POINTS FORTS DU TFE	29
6.2.1	<i>Originalité</i>	29
6.2.2	<i>Plausibilité</i>	29
6.2.3	<i>Chaos</i>	30
6.2.4	<i>Game Feel</i>	30
6.3	FAIBLESSES ET PISTES D'AMELIORATIONS	30
6.3.1	<i>Performances</i>	30
6.3.1.1	Simulation des cordes hors-champ	30
6.3.1.2	Réduction adaptative de la fréquence d'actualisation des particules	31
6.3.1.3	Réduction adaptative du nombre de particules	31
6.3.1.4	Parallélisation des calculs	31
6.3.2	<i>Elasticité indésirée et manque d'interaction physique</i>	31
6.4	PROLONGEMENTS.....	32
6.4.1	<i>Moteur physique unifié</i>	32
6.4.2	<i>Détection de collisions</i>	32
6.4.2.1	Broad Phase.....	32
6.4.2.2	Narrow Phase	33
6.4.3	<i>Système généralisé de contraintes</i>	33
7	CONCLUSION	35
8	BIBLIOGRAPHIE	36
9	ANNEXES	38
9.1	ANNEXE 1 : PORTFOLIO DE R&D.....	38
9.2	ANNEXE 2 : DIAGRAMMES UML	39
9.2.1	<i>Diagramme de cas d'utilisation</i>	39
9.2.2	<i>Diagramme de classes</i>	40
9.2.3	<i>Diagramme de séquence</i>	40
9.2.4	<i>Diagramme d'activités</i>	41
9.3	ANNEXE 3 : PROOF OF CONCEPT	42
9.4	ANNEXE 4 : DEMONSTRATION DE LA TENSION D'UNE CORDE REELLE ENTRE DEUX CORPS SUR UNE SURFACE SANS FRICTION	

TABLE DES ILLUSTRATIONS

EQUATIONS

ÉQUATION 1 : INTEGRATION D'EULER	9
ÉQUATION 2 : INTEGRATION DE VERLET	9

FIGURES

FIGURE 3-1 : CORRECTION DE LA DISTANCE ENTRE DEUX PARTICULES (JAKOBSEN, 2001)	8
FIGURE 3-2 : PSEUDO-CODE RENDANT LE PAS DE TEMPS FIXE	10
FIGURE 3-3 : PSEUDO-CODE DE L'ALGORITHME DE JAKOBSEN (2001)	11
FIGURE 3-4 : PSEUDO-CODE L'ALGORITHME DE PBD (MÜLLER ET AL., 2006)	12
FIGURE 3-5 : PSEUDO-CODE DES MODIFICATIONS DE MÜLLER ET AL. (2006) APPORTEES A LA METHODE PBD	14
FIGURE 3-6 : RESULTATS DE MACKLIN ET AL. (2019). GAUCHE : 1 SUBSTEP, 100 ITERATIONS (4MS/FRAME) - MILIEU : 1 SUBSTEP, 500 ITERATIONS (17MS/FRAME) - DROITE : 100 SUBSTEPS, 1 ITERATION (6MS/FRAME)	14
FIGURE 4-1 : PREMIER PROTOTYPE DU JEU (23-04-24)	15
FIGURE 4-2 : PREMIERE PoC D'UNE CORDE REALISTE (CAYERE, 2024)	16
FIGURE 4-3 : PSEUDO-CODE DE L'ALGORITHME IMPLEMENTE DANS LA SOLUTION	17
FIGURE 4-4 : PSEUDO-CODE DE RESOLUTION DE CONTRAINTE DE DISTANCE ENTRE DEUX PARTICULES	17
FIGURE 4-5 : SCHEMA DE RESOLUTION DE COLLISION D'UNE PARTICULE AVEC UN OBJET	18
FIGURE 5-1 : RENDU DE LA CORDE - HAUT GAUCHE : LACHE - HAUT DROITE : TENDUE - BAS : DANS LE TFE	21
FIGURE 5-2 : TENSION DE LA CORDE - HAUT GAUCHE : TENSION APPLIQUEE SUR DES POINTS D'ACCROCHE DIFFERENTS - HAUT DROITE : SYSTEME DE POIDS AVEC DES OBJETS SUSPENDUS - BAS : OBJETS SUSPENDUS DANS LE TFE	22
FIGURE 5-3 : COLLISION DE LA CORDE - GAUCHE : OBJETS RETENUS PAR LA CORDE - DROITE : COLLISIONS MULTIPLES DANS LE TFE ..	22
FIGURE 5-4 – COMPARAISON DE CORDES AVEC FRICTION ELEVEE (GAUCHE) ET SANS FRICTION (DROITE)	23
FIGURE 5-5 : ACTIONS DU JOUEUR (DE DROITE A GAUCHE ET DE HAUT EN BAS) : S'ACCROCHER, RELIER, COUPER, ALLONGER, RACCOURCIR	23
FIGURE 5-6 : GAUCHE : MERINOS TIRE UN OBJET LEGER – DROITE : RETENU PAR UN OBJET TRES LOURD	24
FIGURE 5-7 : GAUCHE : MERINOS ACCROCHE A UN VENTILATEUR AVEC UNE CORDE LONGUE - DROITE : SUSPENDU AVEC UNE CORDE COURTE	24
FIGURE 5-8 : SYSTEME DE VISEE EN FONCTIONNEMENT, FACILITE POUR LE JOUEUR DE VISER UN LIVRE PLUTOT QUE L'ETAGERE	25
FIGURE 5-9 : AFFICHAGE DES POSITIONS DES PARTICULES EN TEMPS REEL	26
FIGURE 5-10 : SCENE CHAOTIQUE TOURNANT A MOINS DE 20MS/FRAME	27
FIGURE 9-1 : DIAGRAMME DE CAS D'UTILISATION DU SYSTEME DE CORDE	39
FIGURE 9-2 : DIAGRAMME DE CLASSES DU SYSTEME DE CORDE	40
FIGURE 9-3 : DIAGRAMME DE SEQUENCE DU DIALOGUE ENTRE LE JOUEUR ET LE SYSTEME DE CORDE	40
FIGURE 9-4 : DIAGRAMME D'ACTIVITES DE LA FONCTION UPDATE DU SYSTEME DE CORDE	41

1 INTRODUCTION

Dans le cadre de notre travail de fin d'étude du master « Jeu Vidéo » de la HEAJ, nous avons pour consigne de créer un jeu vidéo complet de son idéation à la création de sa page Steam. Après une phase de recherche et plusieurs prototypages, nous nous sommes orientés vers un choix original : un jeu « Physics-Based » (jeu dans lequel la physique est au centre du gameplay) dans lequel le joueur a la capacité de relier tout élément du monde (objets, personnages) à l'aide d'une corde simulée physiquement.

Le choix de cette mécanique implique la simulation physique d'une corde capable de transmettre des forces aux objets qu'elle relie et d'interagir avec son environnement de manière plausible, comme le fait une corde dans le monde réel. De plus, le contexte d'un jeu vidéo nous impose de fournir une solution stable et suffisamment performante pour en assurer sa fluidité sur la plupart des ordinateurs. En outre, le joueur doit trouver l'utilisation de cette corde intuitive, précise et gratifiante.

Afin de satisfaire l'ensemble de ces contraintes, différentes méthodes de simulation sont à envisager et comparer afin de trouver le meilleur compromis entre performance, stabilité, plausibilité et contrôle de la simulation. Une fois ce choix fait et implémenté, il doit être confronté aux joueurs pour en ajuster les paramètres et en améliorer son ergonomie.

Ce sujet a captivé ma curiosité par sa complexité et les perspectives qu'il ouvre pour d'autres projets, tant sur le plan théorique que pratique. Ainsi, le but de ce travail est de développer une solution de simulation de corde. Ce mémoire en présente le processus de recherche, de développement et d'intégration dans le TFE.

2 PROBLEMATIQUE

La simulation de corde dans l'environnement dynamique d'un jeu vidéo représente un défi technique complexe. En effet, il s'agit de concilier la plausibilité de la simulation (transmission des forces, collisions, tension, gravité), la stabilité de la solution et les contraintes de performances inhérentes au jeu vidéo (maintien d'un framerate stable à 60 FPS).

De plus, la corde étant la mécanique principale de notre TFE, elle doit dépasser le simple cadre technique de la simulation physique. Son utilisation doit être fluide, intuitive et plaisante pour le joueur qui doit la percevoir comme un outil de jeu plutôt qu'une contrainte.

Ce travail doit également respecter quelques contraintes imposées par les consignes du TFE dans lequel il s'intègre. Il doit être réalisé dans le moteur de jeu Unreal Engine 5.4 et le jeu dont il découle doit être en trois dimensions.

Dans ce contexte, nous cherchons à répondre à deux questions complémentaires :

1 - Dans le cadre d'un jeu vidéo, comment simuler de manière crédible et fluide des cordes dans un environnement 3D dynamique ?

2 - Quels ajustements créatifs permettent de rendre l'utilisation de ces cordes intuitive et satisfaisante pour le joueur ?

3 ÉTAT DE L'ART

3.1 RAPPELS ET NOTIONS ELEMENTAIRES

3.1.1 Simulation d'une corde

La méthode la plus commune pour simuler une corde, présentée par Jakobsen (2001) consiste à subdiviser la corde en un ensemble de particules. Chacune d'entre elle est reliée à ses voisines par une contrainte de distance définie. A chaque pas de temps, après avoir intégré leur position, on itère sur toutes les particules et on vient rectifier deux à deux leur position pour satisfaire la contrainte de distance correspondante. Ainsi, on éloigne ou on rapproche les deux particules l'une de l'autre en fonction de leur écartement actuel pour atteindre cette distance idéale (*Figure 3-1*).

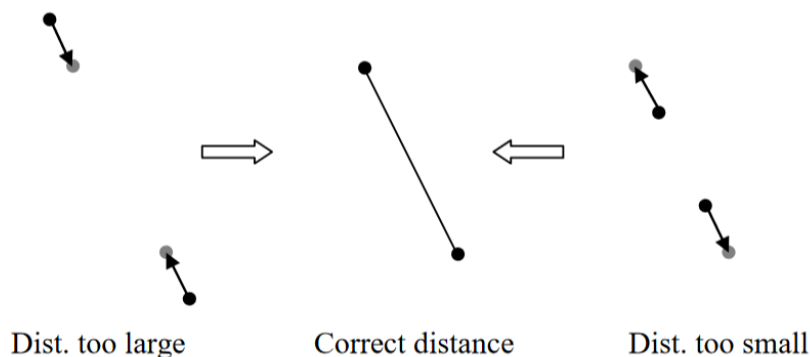


Figure 3-1 : Correction de la distance entre deux particules (Jakobsen, 2001)

3.1.2 Intégration

En physique, la cinématique permet de décrire comment un corps se déplace dans l'espace et dans le temps sans se préoccuper des causes de ce mouvement. La dynamique est la théorie des causes de ces mouvements, elle fait appel au concept de force pour expliquer le mouvement des corps (Benson, 2015). Cette théorie permet d'introduire les notions de friction, collision, rebond, etc. Ces paramètres doivent être pris en considération lors de l'élaboration du projet dans un but de réalisme.

De plus, dans un jeu vidéo, de nouvelles contraintes viennent s'ajouter. Bien souvent, le joueur est capable d'interagir avec son environnement, soit directement par les actions que lui permettent les designers du jeu, soit indirectement par sa position dans le jeu. En effet, pour des raisons d'optimisation, on cherche à simuler uniquement la physique des objets que le joueur peut voir et à désactiver celle des autres. Enfin, contrairement au temps réel qui lui est continue, nos calculs physiques se basent sur un temps discret (de l'ordre de 60 frames par seconde, soit 0.167s dans des conditions optimales) et bien souvent variable.

Au vu de ces contraintes, le calcul exact de l'ensemble de ces forces est impossible en temps réel. Nous devons donc faire des approximations de calcul des positions et des vitesses. Dans ce projet, nous utilisons deux méthodes d'approximations de ces calculs.

3.1.2.1 Méthode d'Euler

La méthode d'Euler est certainement la plus connue en raison de son intuitivité et de sa facilité d'implémentation. C'est une méthode explicite, c'est-à-dire qu'elle a besoin d'une première position pour calculer la position suivante d'un objet et ne peut donc pas être réversible (Roussel, 2015). Cette équation se présente sous la forme :

$$\begin{aligned}x' &= x + v \cdot \Delta t \\v' &= v + a \cdot \Delta t\end{aligned}$$

Équation 1 : Intégration d'Euler

Où x' et x sont respectivement les positions suivante et actuelle ; v' et v les vitesses suivante et actuelle ; Δt le pas de temps ; a l'accélération de l'objet calculée grâce à la seconde loi de Newton $f = m \cdot a$ (f correspondant à l'accumulation des forces appliquées à la particule et m sa masse).

Cette méthode, bien que simple d'utilisation, a le gros défaut d'introduire de l'énergie dans le système et présente un niveau d'instabilité proportionnel au pas de temps choisi, bafouant le principe de conservation de l'énergie. Dans le cas des jeux vidéo, le pas de temps utilisé est important (0.0167s à 60 fps). De ce fait, cette méthode a tendance à propager et même à amplifier les erreurs de calculs. Enfin, dans certaines situations (i.e. lors de la résolution de collisions), les calculs se complexifient fortement puisque nous devons corriger la vitesse de l'objet de sorte que la position soit parfaitement correcte malgré tous les facteurs extérieurs.

Dans le cas de la simulation d'une corde, chacune des particules qui la constitue propage l'erreur d'approximation de sa position à ses voisines, ce qui peut conduire à un système très instable et explosif.

3.1.2.2 Méthode de Verlet

La méthode de Verlet, quant à elle, ne s'appuie plus sur le calcul de la vitesse pour calculer la nouvelle position d'un corps. En effet, celle-ci est représentée implicitement par la différence entre les deux positions précédentes (Roussel, 2014). Elle se présente sous la forme :

$$\begin{aligned}x' &= 2x - x^* + a \cdot \Delta t^2 \\x^* &= x\end{aligned}$$

Équation 2 : Intégration de Verlet

Où x' , x et x^* sont respectivement les positions suivante, actuelle, et précédente ; Δt est le pas de temps et a l'accélération.

Cette équation règle plusieurs soucis de la méthode d'Euler. Comme la vitesse est déterminée implicitement par une différence de positions, la désynchronisation de la position avec la vitesse n'est pas possible. Les erreurs de résolution de collisions et de contraintes de distance ne sont donc pas amplifiées ce qui offre un système bien plus stable (Jakobsen, 2001).

C'est la méthode d'intégration privilégiée lorsque l'on souhaite simuler un système de particules comme un fluide, un tissu, ou bien encore une corde (Müller et al., 2006).

Quelques problèmes subsistent néanmoins. Comme les vitesses des objets ne sont pas stockées il est très difficile d'appliquer des forces de friction, d'amortissement ou de rebond. De plus, cette méthode d'intégration requiert un pas de temps constant. Dans le cadre d'un jeu vidéo, l'utilisation du temps

d'une frame (variable *DeltaTime* dans Unreal Engine) est donc proscrit, puisque variable. Nous devons donc forcer un pas de temps constant défini en amont, et subdiviser le *DeltaTime* par ce pas de temps (Figure 3-2). Cette solution est utilisée dans une méthode d'optimisation de substepping (Macklin et al., 2019), présentée plus loin (c.f. 3.4.1).

```
1  main loop
2      RemainingTime += DeltaTime
3      while RemainingTime ≥ SubstepTime
4          RemainingTime -= SubstepTime
5          integrate()
6      endwhile
7  endloop
```

Figure 3-2 : Pseudo-code rendant le pas de temps fixe

3.2 CONFRONTATION DES APPROCHES DE SIMULATION

Il existe trois principales méthodes de simulation, chacune avec leur lot d'avantages et d'inconvénients. Il s'agit alors de choisir la bonne au regard du contexte dans lequel on veut les utiliser. Dans notre cas, la précision physique de notre simulation n'est pas la priorité, tant que la solution semble crédible aux yeux du joueur. Les deux points importants vont alors être la rapidité d'exécution et la stabilité du système (Jakobsen, 2001).

3.2.1 Approche basée sur des forces

Cette approche, traditionnellement utilisée en physique, est de loin la plus précise, mais aussi la plus couteuse. En effet, à chaque pas de temps, on calcule la somme des forces internes (élasticité, viscosité, etc.) et externes (gravité, collisions, tensions, etc.) des corps. Puis, par l'application de la deuxième loi de Newton, on calcule leur accélération. Enfin on obtient la vitesse puis la position des corps du système (Müller et al., 2006).

Pour des logiciels nécessitant une exécution en temps réel comme dans notre cas, cette solution ne semble pas optimale puisque la précision n'est pas notre principale contrainte. La lourdeur des calculs et les deux intégrations à faire par frame et par objets ne semblent pas nécessaire à la simulation crédible d'un jeu. De plus, lorsqu'une collision entre deux objets survient, cette approche ne résout les positions qu'à la frame suivante, entraînant une réaction différée des corrections des positions des objets. Ainsi, due à cette frame de latence, la correction des positions est parfois surestimée et entraîne des réactions instables et explosives (Müller et al., 2006).

3.2.2 Approche basée sur des vitesses

Cette approche, fort utilisée dans la plupart des moteurs physiques simulant des corps solides (Rigid Bodies) manipulent directement les vitesses et s'émancipent du calcul des forces des objets. Il en résulte un système bien moins lourd et plus stable que l'approche précédente lorsqu'il s'agit de résoudre des collisions. Cependant, un problème de glissement survient et sa correction nécessite des ajustements supplémentaires plus complexe (Müller et al., 2006).

Dans des systèmes fortement contraints, comme les simulations de particules (cordes, tissus, fluides, Soft Bodies, etc.), il devient difficile d'obtenir une solution stable, rapide et crédible à la fois. Les erreurs s'accumulent vite et rendent le système instable.

3.2.3 Approche basée sur les positions

La dernière approche, consiste à agir directement sur les positions des objets, abstrayant les forces et les vitesses (Müller et al., 2006). Cette solution, beaucoup moins coûteuse et bien plus stable, est l'approche privilégiée pour la simulation de système de particules comme celui de la corde de ce mémoire. Elle offre cependant une précision de calcul moins juste que les deux approches précédentes.

Cette approche est utilisée dans le cadre de ce travail et des versions de son implémentation sont décrites ci-dessous.

3.2.3.1 Première version

Jakobsen (2001) est le premier dans la littérature à proposer cette approche dans le domaine du jeu vidéo. Il propose un ensemble de techniques permettant de simuler la physique des personnages dans Hitman : Codename 47 de manière plausible, stable et optimisée.

```
1  main loop
2  forall particles
3      temp =  $x_{n-1}$ 
4       $x_n += x_n - x_{n-1} + g \times \Delta t^2$ 
5      p = temp
6  endfor
7  loop solverIterations times
8      satisfyConstraints()
9  endloop
10 endloop
```

Figure 3-3 : Pseudo-code de l'algorithme de Jakobsen (2001)

Dans sa solution (Figure 3-3), les personnages sont représentés par des particules reliés par des contraintes de distance. Ainsi, à chaque pas de temps, il intègre la position de ces particules (lignes 2 à 6) avec une intégration de Verlet (c.f. 3.1.2.1 ci-dessus). Puis, il itère sur toutes les contraintes de distance (c.f. 3.1.1 ci-dessus) et rectifie la position des particules (ligne 8).

Pour résoudre les contraintes de collision, Jakobsen (2001) utilise une approche très simple : il se contente de projeter les positions des particules hors des éléments avec lesquels elles sont entrées en collision. Cette technique de projection fonctionne en synergie avec le calcul implicite de la vitesse des particules. En effet, comme la position suivante d'une particule se base sur ses deux positions précédentes, les changements de vitesse se font implicitement.

Enfin, comme plusieurs contraintes sont résolues successivement, il arrive que la résolution de l'une entraîne l'invalidité de l'autre. Pour remédier à ce problème, Jakobsen, (2001) utilise la méthode de relaxation détaillée par Press et al. (2007) qui consiste à résoudre consécutivement les différentes contraintes d'un objet et de réitérer l'opération un certain nombre de fois (lignes 7 à 9). La solution va alors converger vers une configuration idéale satisfaisant toutes les contraintes.

3.2.3.2 PBD

Müller et al. (2006), chercheurs chez Nvidia, ont développé et amélioré ce système pour créer le système PBD (Position Based Dynamics). Il fonctionne de manière similaire au système pensé par Jakobsen (2001), à quelques différences près.

Le problème soulevé par Müller et al. (2006) inhérent à la solution de Jakobsen (2001) est sa difficulté à calculer les forces de friction, d'amortissement (résistance de l'air, frottements internes du système) et de restitution. En effet, Jakobsen (2001) utilise ces particules pour simuler la physique des personnages d'Hitman et n'avait donc pas besoin de tels calculs. Or dans le cas de tissus, de cordes ou même de Rigid Bodies, lorsqu'on essaie de généraliser la solution, on est confronté à ce problème. Pour le résoudre, Müller et al. (2006) utilisent conjointement les méthodes d'intégration d'Euler (c.f. 3.1.2.1) et de Verlet (c.f. 3.1.2.2) pour stocker et manipuler la vitesse.

```

1  main loop
2    forall particles do     $v \mathrel{+}= g \times \Delta t$ 
3    dampVelocities()
4    forall particles do     $x_n = x_{n-1} + v \times \Delta t$ 
5    forall particles do    GenerateCollisionConstraints()
6    loop solverIterations times
7      projectConstraints()
8    endloop
9    forall particles do
10       $v = (x_n - x_{n-1}) / \Delta t$ 
11       $x = p$ 
12    endfor
13    velocityUpdate()
14  endloop

```

Figure 3-4 : Pseudo-code l'algorithme de PBD (Müller et al., 2006)

Au début de chaque pas (Figure 3-4), la vitesse est intégrée puis corrigée selon une valeur d'amortissement (damping) définie par l'utilisateur (ligne 2-3). La position est alors intégrée selon la méthode d'Euler (ligne 4).

Une nouvelle étape est ajoutée (ligne 5) pour optimiser le calcul des collisions. C'est la phase de génération de paires de collision (Broad Phase), permettant d'éviter de les recalculer inutilement plusieurs fois par frame dans la boucle de relaxation (ligne 6 à 8) également présente dans la solution de (Jakobsen, 2001).

Enfin la vitesse est recalculée selon la méthode de Verlet (ligne 9 à 12) puis est corrigée selon les forces résultantes des collisions et des contraintes calculées plus haute.

3.3 CABLE COMPONENT D'UNREAL ENGINE

Notre projet a pour contrainte l'utilisation du moteur de jeu Unreal Engine. Une solution est proposée nativement par ce moteur pour la simulation de cordes : c'est le plugin Cable Component (Golding, 2014). Ce plugin met en application la solution de Jakobsen (2001) et propose quelques options permettant de personnaliser le comportement des cordes. Cependant, les cordes générées par ce plugin sont assez limitées et ne sont pas capables de transmettre des forces aux objets qu'elles relient ou qu'elles touchent. On ne peut pas non plus les couper et peu de solutions sont mises en place pour optimiser le code. Enfin le système de collisions est quelque peu approximatif, laissant les cordes passées à travers d'autres objets dans certaines situations (Cayèré, 2024).

Cette solution n'est donc pas suffisante au regard des attentes du projet. Cependant, la méthode utilisée pour générer le mesh est performante et utilise le framework de rendu d'Unreal Engine (Golding, 2014). La corde faite dans le cadre de ce mémoire en est donc fortement inspirée sur cette partie.

Pour une explication plus détaillée de cette la comparaison d'une solution personnalisée et du Cable Component d'Unreal Engine, se référer à l'*Annexe 1 : Portfolio de R&D*.

3.4 AMELIORATIONS DU SYSTEME

3.4.1 Small steps

Macklin et al. (2019), propose une amélioration simple de PBD. Ils constatent que l'utilisation d'un substep (sous-pas) de temps plus petit est bien plus performant que l'itération successive des résolutions de contraintes.

```
1  main loop
2  DetectCollisionPairs()
3   $\Delta t_g = \frac{\Delta t_f}{n_{steps}}$ 
4  while  $n < n_{steps}$ 
5  > ... ..
9  loop solve iterations times
10  solveConstraints()
11  endloop
12  > ... ..
16  velocityUpdate()
17   $n += 1$ 
18  endwhile
19  endloop
```

Figure 3-5 : Pseudo-code des modifications de Müller et al. (2006) apportées à la méthode PBD

Cette découverte (Figure 3-5) permet de supprimer la boucle de résolution des contraintes (ligne 9 et 11). Celle-ci est troqué par une autre qui permet de subdiviser le pas de temps actuel en un certain nombre de substeps plus petits (lignes 3-4). Pour des questions d'optimisation, la phase de détection de collisions est sortie de cette boucle. Cette phase consiste en la détection de paires d'objets potentiellement superposés.

Par la suite nous utiliserons une version modifiée de cet algorithme qui consiste à définir directement la valeur du substep et non le nombre d'itérations qui est alors induit par la valeur de Delta Time (c.f. 3.1.1 Simulation d'une corde). Cette modification permet d'avoir une solution qui n'est pas dépendante du framerate (Figure 3-6).

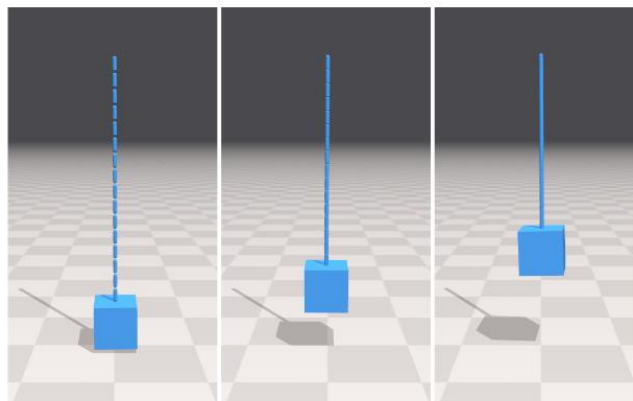


Figure 3-6 : Résultats de Macklin et al. (2019). Gauche : 1 substep, 100 iterations (4ms/frame) - Milieu : 1 substep, 500 itérations (17ms/frame) - Droite : 100 substeps, 1 itération (6ms/frame)

4 METHODOLOGIE

Nous présentons ici les différentes étapes de la méthodologie suivie pour la réalisation de ce mémoire. Comme expliqué dans l'introduction, ce travail a pour but d'être intégré comme mécanique principale du TFE. Il est donc nécessaire de concilier la rigueur de la recherche avec le besoin de playtests intrinsèque au milieu du jeu vidéo. La solution doit être éprouvée le plus souvent possible et directement en jeu pour que les joueurs se fassent une idée de la direction que nous prenions, nous permettant de valider, infirmer ou transformer le choix de cette mécanique au besoin. Ainsi, il est donc nécessaire de travailler en suivant une approche itérative, ponctuée de playtests permettant d'adapter la mécanique aux besoins du joueur.

4.1 PREMIER PROTOTYPE

Rapidement, nous avons eu besoin de créer un prototype de cette corde pour éprouver notre choix de design auprès des joueurs. Cette contrainte de temps a poussé au développement d'une solution minimaliste (*Figure 4-1*) ne présentant que les aspects primordiaux de la mécanique : appliquer une contrainte de distance entre deux corps (deux objets, ou un objet et le personnage), et couper cette contrainte quand nécessaire.

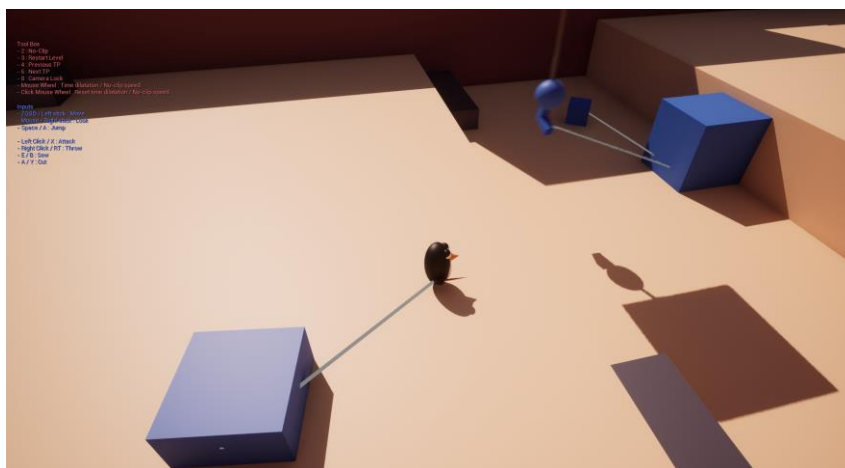


Figure 4-1 : Premier prototype du jeu (23-04-24)

Les playtests qui ont suivi ont permis de valider ce choix et il a été alors possible de creuser plus en profondeur le sujet pour rendre cette proto corde plus plausible et agréable à utiliser.

4.2 POC DE LA CORDE

Dans des projets à part du TFE, deux « Proofs of Concept » (PoC) de cordes réalistes ont été réalisées et développées en C++ avec le moteur de jeu Unreal Engine. La première s'inscrit dans le cadre du projet de Recherche et Développement de la première année du master « Jeux Vidéo » de la HEAJ.

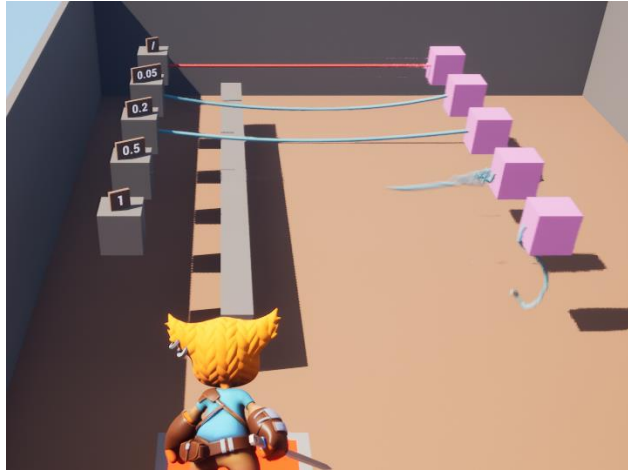


Figure 4-2 : Première PoC d'une corde réaliste (Cayère, 2024)

L'objectif de cette première PoC (Figure 4-2) était de développer une première implémentation d'une corde réaliste et de comparer cette simulation personnalisée avec le Cable Component d'Unreal Engine. Pour les détails de cette implémentation, se référer à l'Annexe 1 : *Portfolio de R&D*.

Cette année, une deuxième PoC a été réalisée pour mettre en évidence les éléments essentiels à la simulation de la corde. Tout le code de la solution est ainsi extrait du mémoire pour en faciliter sa lecture. Il est disponible dans l'Annexe 3 : *Proof of Concept*.

4.2.1 Conception

Le travail a débuté par la conception de l'architecture de la solution. Celle-ci consiste en une classe mère en C++ responsable de la simulation en tant que telle tandis qu'une classe fille est responsable des feedbacks visuels et facilite la manipulation des valeurs par défaut pour les Game Designer. Toutes les cordes sont gérées par un Rope Manager (c.f. *Rope Manager*), référencé par le Game Mode, qui s'occupe du dialogue entre les autres acteurs et ces cordes. En outre, la corde utilise deux structures de données représentant pour l'une les particules de la corde et pour l'autre les points d'accroche de ses deux extrémités (c.f. Annexe 2 : *Diagrammes UML*).

4.2.2 Algorithme d'actualisation

Après avoir créé les différentes structures de données, l'étape suivante est de développer l'algorithme d'actualisation des cordes à chaque frame (fonction Tick dans Unreal Engine). L'algorithme PBD (c.f. *PBD*) de Müller et al. (2006) a été adapté pour convenir à nos besoins.


```

1  main loop
2    CollidingParticles = collectCollidingParticles()
3    RemainingTime += DeltaTime
4    while RemainingTime ≥ SubstepTime
5      RemainingTime -= SubstepTime
6      integrate()
7      performDistanceSolve()
8      performCollisionSolve(CollidingParticles)
9      updateVelocities()
10   endwhile
11   applyForces()
12 endloop

```

Figure 4-3 : Pseudo-code de l'algorithme implémenté dans la solution

En premier lieu (Figure 3-1), nous avons appliqué la technique de Small Steps (c.f. 4.5.23.4.1) de (Macklin et al., 2019) en la modifiant légèrement (lignes 3 à 5) pour avoir un Substep constant (Figure 3-2). Il a également été ajouté une phase d'actualisation des forces des objets reliés (ligne 11) qui consiste à faire le transfert de force entre le système de corde et le système physique natif d'Unreal Engine. Enfin une phase de Broad Phase a été également mise en place pour des questions d'optimisation (c.f. 4.5.3).

4.2.3 Contrainte de distance

La phase de résolution de contrainte de distance implémentée ici est directement issue du papier de Jakobsen (2001). Comme expliqué plus haut (c.f. 3.1.1), chaque particule a une contrainte de distance avec une ou deux voisines. Pour résoudre cette contrainte (Figure 4-4), on vient simplement les rapprocher ou les éloigner en fonction de la différence (ligne 2 à 4) entre l'écart actuel des deux particules et la distance de « repos » souhaitée (défini lors de la création de la corde). Cette correction est pondérée par l'inverse de la masse des deux particules (ligne 5 et 6). Fonctionner avec l'inverse des masses permet facilement de relier des objets à des objets statiques. En effet, l'inverse d'une masse infini (masse d'un objet statique) vaut zéro, ne modifiant la position que de l'autre particule.

```

1  function performDistanceSolve()
2    Delta = Particle2.Pos - Particle1.Pos
3    CorrectionLength = ||Delta|| - RestLength
4    CorrectionVector = CorrectionLength × Normalize(Delta)
5    Particle1.Pos += CorrectionVector ×  $\frac{Particle1.InvMass}{Particle1.InvMass + Particle2.InvMass}$ 
6    Particle2.Pos -= CorrectionVector ×  $\frac{Particle2.InvMass}{Particle1.InvMass + Particle2.InvMass}$ 
7  endfunction

```

Figure 4-4 : Pseudo-code de résolution de contrainte de distance entre deux particules

4.2.4 Collision et friction

La phase de résolution de collision consiste à effectuer un test de collision entre la position précédente et la position actuelle de chaque particule de la corde. Pour ce faire, on utilise la fonction « SphereTraceSingle » d'Unreal Engine qui renvoie les informations de collisions (profondeur, position de l'impact, objet percuté, etc.) si elle a eu lieu. On projette alors la position de la particule le long de la normale de l'impact d'une distance égale au rayon de la corde, gonflée d'une petite marge de 0.01cm permettant d'éviter aux particules de coller aux surfaces. Enfin, on projette la position actuelle le long de la surface qu'on réduit par un facteur de friction (Benson, 2015) déterminé par l'utilisateur. On actualise finalement la position de la particule à cette position.

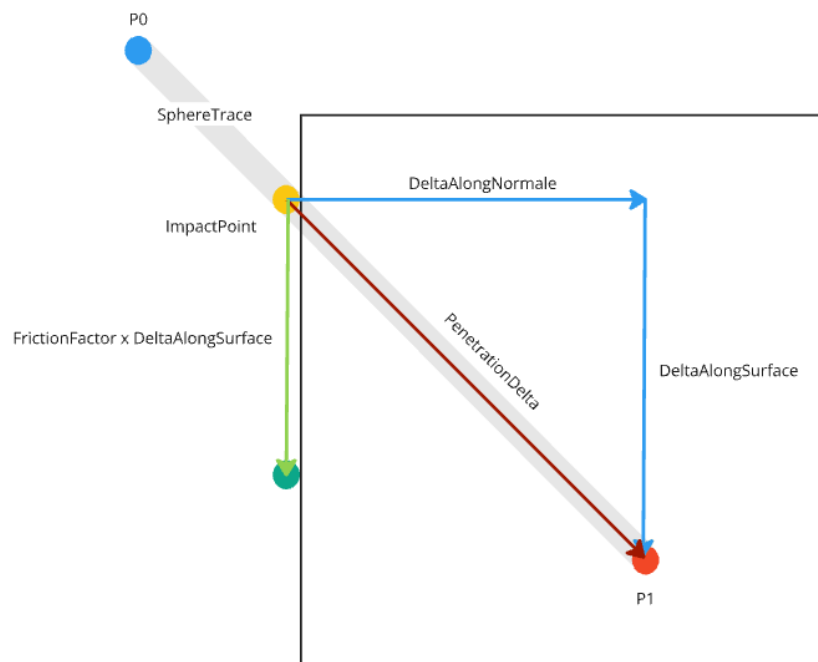


Figure 4-5 : Schéma de résolution de collision d'une particule avec un objet

4.2.5 Interface de design

Afin de permettre un meilleur contrôle sur la création et la gestion des cordes déjà placées dans les niveaux, des outils ont été mis en place. L'objectif ici est de faciliter aux Game Designers l'utilisation de la solution dans l'éditeur.

En effet, il est nécessaire de pouvoir placer des cordes précisément dans les niveaux. Pour ce faire, on utilise deux points représentant les extrémités de la corde. Il est également nécessaire de définir les objets que la corde relie. Placer les points ne suffit pas, nous devons donc transmettre ces informations à la corde en lui pointant l'Actor et le Component liés à chacune des deux extrémités. Cela nous permet notamment de suspendre des objets, maintenir des portes fermées, etc.

Enfin, le comportement de chaque corde doit pouvoir être personnalisé indépendamment des autres. Cela nous permet de faire des cordes avec des visuels différents et des comportements différents (plus de rigidité, pas de collisions calculées, etc.)

4.3 INTEGRATION DE LA SOLUTION DANS LE TFE

Une fois la PoC réalisée, nous avons procédé à son implémentation au sein du TFE. Nous avons besoin de retours des joueurs sur cette nouvelle version de la corde qui transforme l'expérience de jeu en quelque chose de beaucoup plus dynamique.

4.3.1 Actions du joueur sur la corde

Avant de pouvoir faire tester cette nouvelle version, il fallait permettre aux joueurs de la manipuler à l'aide d'actions qu'on lui exposait. Ainsi nous avons permis au joueur de relier son personnage à un objet, d'ensuite relier deux objets ensemble, de rallonger et raccourcir sa corde, ou encore de couper les cordes lorsqu'il le souhaite.

4.3.2 Nouveau système de rendu

Dans le premier PoC réalisé (c.f. *PoC de la corde*), la corde était représentée par un ensemble de cylindre (Rigid Bodies), chacun correspondant à un segment. Cela posait un gros problème de performance étant donné qu'une corde moyenne pouvait compter entre 50 et 100 particules. La possibilité de la rallonger ou de la raccourcir impliquait de générer un pool maximum de cylindre à sa création, entraînant des piques de lag lors de la création de nouvelles cordes. Nous nous sommes alors fortement inspirés de la méthode utilisée par le Cable Component d'Unreal (c.f. 3.3) pour alléger considérablement les temps de calculs inutiles et faire en sorte que la corde ne soit représentée que par un unique mesh.

4.3.3 Système de visée automatique

En raison du choix de l'équipe de ne pas laisser le contrôle de la caméra au joueur et de l'environnement dynamique de notre jeu, il a été nécessaire de créer un système de visée automatique des objets à relier. Ce système se base sur le calcul d'un score pour chacun des objets les plus proches. L'objet avec le plus haut score se voit enveloppé d'un contour en surbrillance et un point indique au joueur la position du point d'accroche s'il décide de relier l'objet. Ce score se base sur un ensemble de paramètres qui permettent de deviner le plus justement possible l'objet que le joueur veut relier :

- La direction de l'objet par rapport à celle du « forward » du personnage (avec un forward incliné vers le haut lorsque le joueur saute).
- L'éloignement de l'objet au personnage.
- Un gros malus si l'objet est déjà relié au personnage afin d'éviter qu'il relie un objet à lui involontairement.
- Un modificateur de score généré automatique sur base du volume de l'objet (plus il est petit, plus le score est important) et qui peut être outrepasser par les game-designers lorsque nécessaire (pour des objets de quête par exemple).

4.4 PLAYTESTS ET AJUSTEMENTS

Les retours des joueurs ne se sont pas fait attendre et étaient plutôt encourageants. Cependant, les sensations de jeu (Game Feel) n'étaient pas encore satisfaisantes et nécessitaient davantage d'ajustements du comportement de la simulation. Une attention particulière a donc été portée sur les

actions de manipulation de la corde afin d'en améliorer l'ergonomie. Nous avons cherché à rendre la mécanique la plus intuitive possible pour le joueur.

4.5 OPTIMISATION

4.5.1 Rope Manager

La création d'un « Rope Manager », à savoir une classe responsable de la gestion des cordes (création, actualisation, et destruction), s'est imposée. Ce manager est créé par le gamemode et stocke dans un tableau l'ensemble des cordes déjà placées. Au cours du jeu, chaque création et destruction de corde se fera par son intermédiaire. Enfin, c'est lui qui appelle la fonction Update de chaque corde. Cela nous permettra d'effectuer facilement des optimisations de calculs sur les cordes éloignées et/ou cachées du joueur (c.f. 6.3.1).

4.5.2 Substepping

Pour améliorer les performances et réduire l'élasticité involontaire des cordes, la méthode de Small Steps (c.f. *Small steps*) présenté par Macklin et al (2019), a été implémentée. Ainsi, nous avons pu améliorer la stabilité de la solution tout en réduisant le temps de calcul.

4.5.3 Broad phase

Dans une simulation physique, le point névralgique est systématiquement la résolution des collisions. Habituellement, les moteurs de jeux séparent la résolution des collisions en deux phases distinctes (Müller, 2025) :

- La Broad Phase : dans cette phase on abstrait chaque objet par des Aligned Axis Bounding Boxes (AABB) et on cherche à récupérer toutes les paires d'objets superposées.
- La Narrow Phase : cette phase permet de déterminer s'il y a bien eu une collision entre deux objets et, le cas échéant, de corriger leur position et/ou leur vitesse.

Dans la première PoC (c.f. *PoC de la corde*), les collisions étaient calculées pour chaque particule de toutes les cordes plusieurs fois par frame, même si les particules en question étaient très éloignées de tout objet. Ainsi, avant la phase de substepping, nous avons adaptée une phase de Broad Phase.

Dans cette phase, nous venons simplement faire le même test de collision que dans la phase de résolution des collisions (c.f. 4.2.4). On utilise ainsi la fonction « SphereTraceSingle » d'Unreal mais, au lieu de se baser sur la position actuelle et précédente de la particule au sein d'un substep, on se base sur sa position actuelle et une projection de sa position à la fin de la frame. Ainsi, si aucune collision n'est détectée, la particule est exclue de la phase de résolution de collision pour la frame en cours.

5 PRESENTATION DU PROTOTYPE

Dans cette section nous présentons les résultats issus de la solution développée. Les figures de cette partie sont issues soit de la PoC qui met en meilleur évidence certains comportements, soit directement du TFE. Le projet de la PoC est disponible dans l'*Annexe 3 : Proof of Concept*.

5.1 COMPORTEMENT ET SIMULATION

5.1.1 Rendu et gravité

Sur le plan visuel, la corde ressemble et se comporte de manière plausible, permettant ainsi d'atteindre un des objectifs de ce travail. En effet, lorsque les deux extrémités se rapprochent, le reste de la corde pend sous l'effet de la gravité. A contrario, lorsque les deux points s'éloignent l'un de l'autre, la corde se tend.

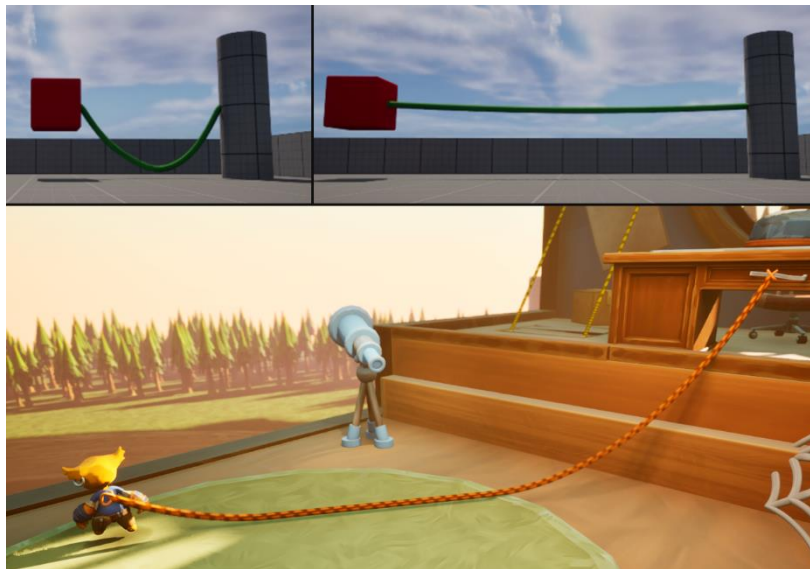


Figure 5-1 : Rendu de la corde - Haut gauche : lâche - Haut droite : tendue - Bas : Dans le TFE

5.1.2 Tension

La corde mise en place permet de répercuter les forces d'un objet sur un autre comme attendu. La position du point d'accroche relative à l'objet attaché est déterminante sur la simulation de la tension. En effet, tirer un cube depuis le centre d'une face n'a pas le même effet que de le tirer depuis un de ses angles. Le poids des objets est également significatif dans son comportement. Lorsqu'un objet lourd et un objet léger sont reliés ensemble et que la corde est tendue, le mouvement de l'objet lourd sera bien plus impactant sur l'objet léger que l'inverse.



Figure 5-2 : Tension de la corde - Haut gauche : tension appliquée sur des points d'accroche différents - Haut droite : système de poids avec des objets suspendus - Bas : Objets suspendus dans le TFE

5.1.3 Collision et friction

La corde est capable de simuler des collisions et réagit aux objets qui entrent en contact avec elle. Si des objets sont reliés aux extrémités de la corde, les forces qu'ils subiront seront ajustées correctement et seront comme attendu.



Figure 5-3 : Collision de la corde - Gauche : Objets retenus par la corde - Droite : collisions multiples dans le TFE

Pour augmenter la plausibilité de la solution, les collisions prennent en compte la friction de contact. Ainsi, elle ne glisse pas perpétuellement et ralenti progressivement en fonction d'une valeur de friction définie par l'utilisateur (Game Designer).



Figure 5-4 – Comparaison de cordes avec friction élevée (gauche) et sans friction (droite)

5.2 ACTIONS DU JOUEUR

5.2.1 Actions de base

La corde faisant partie intégrante de la key-feature de notre TFE, la simulation et les interactions entre la corde et l'environnement ne suffisent pas. Le joueur est également capable de faire un certain nombre d'actions de manipulation de la corde lui permettant de la contrôler à sa guise : créer une corde entre son personnage et un objet, relier deux objets entre eux, couper sa propre corde, couper toutes les cordes dans un certain rayon ou encore rallonger et raccourcir sa corde.

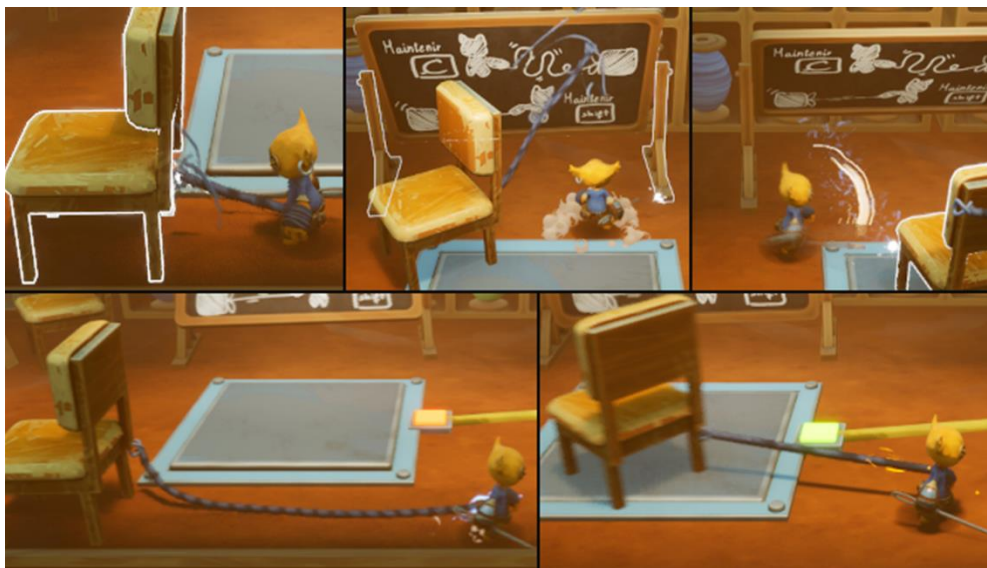


Figure 5-5 : Actions du joueur (de droite à gauche et de haut en bas) : S'accrocher, relier, couper, allonger, raccourcir

5.2.2 Manipulation des objets reliés

Lorsqu'il est relié à un objet suffisamment léger, le joueur peut le tirer en se déplaçant ou en raccourcissant sa corde. En revanche, s'il est trop lourd, le joueur est retenu (et dans certains cas suspendu) à l'objet.

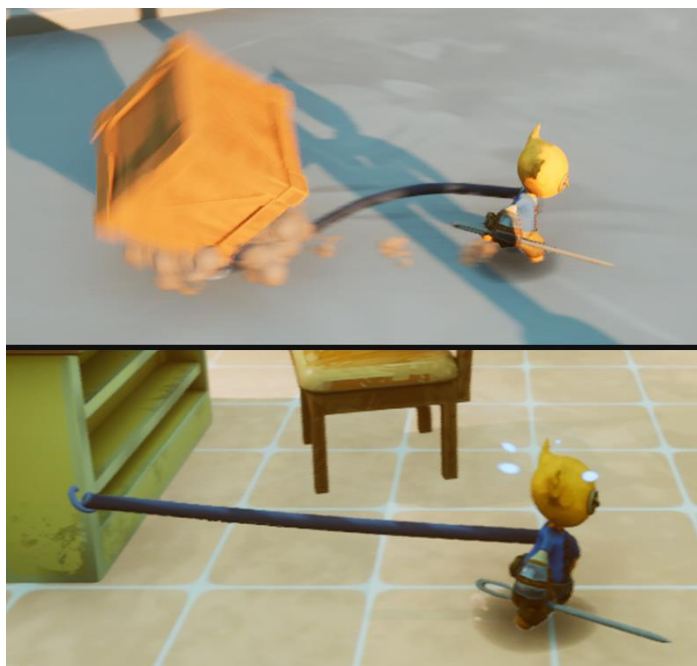


Figure 5-6 : Gauche : Merinos tire un objet léger – Droite : Retenu par un objet très lourd

Ainsi, lorsque le joueur est relié à un objet trop lourd, raccourcir la corde le fait se rapprocher de l'objet et peut dans certains cas le faire se suspendre, lui permettant de passer des obstacles ou d'accéder à des lieux inaccessibles autrement.



Figure 5-7 : Gauche : Merinos accroché à un ventilateur avec une corde longue - Droite : Suspendu avec une corde courte

5.2.3 QoL Improvements

Comme mentionné plus haut (c.f. 4.3.3), la manipulation de la corde par le joueur est une action complexe. Les objets alentours sont autant d'obstacles à la progression du joueur. Même si c'est en partie ce qui fait le sel de notre jeu, nous devons nous assurer que ces perturbations ne paraissent pas injustes aux yeux du joueur. Ainsi nous avons cherché à faciliter au maximum l'utilisation de cette mécanique (Quality of Life Improvements).

5.2.3.1 Système de visée

La première amélioration a donc été la mise en place d'un système de visée « intelligent ». Avant cela, il était très difficile de relier des petits objets proches ou contenus dans des grands (des livres dans une étagère, un CD sur une commode, une peluche sous un lit). Etant donné que les quêtes du jeu attendaient de telles actions du joueur, cette amélioration est vitale pour réduire sa frustration. Depuis sa mise en place, les joueurs ne se plaignent plus de la visée du jeu.

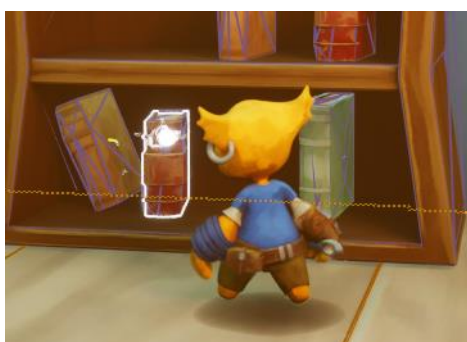


Figure 5-8 : Système de visée en fonctionnement, facilité pour le joueur de viser un livre plutôt que l'étagère

5.2.3.2 Système de téléportation d'objets reliés

Certains raccourcis ont été créés pour faciliter les allers et venues du joueur dans le niveau. Dans ces passages, les objets avaient tendance à rester coincer. Un système de détection a été mis en place permettant de téléporter ces objets au plus proche du joueur. Par exemple, quand le joueur saute dans la cheminée avec la clé reliée, elle se téléporte automatiquement pour éviter de rester coincée dans le grenier.

5.2.3.3 Facilitation des actions de base

Les actions du joueur ont eu de notre part une attention particulière. En effet, étant donné la complexité de notre mécanique, nous devons rendre son utilisation la plus instinctive possible. Ainsi, en observant les joueurs, nous avons cherché à anticiper leurs intentions en améliorant le fonctionnement des actions liées à la corde. En voici trois exemples :

- L'action de « couper » est contextuelle, c'est-à-dire qu'en fonction de l'état du personnage (relié à un objet ou non), cette action va soit uniquement couper le fil qui le relie à un objet, soit déclencher un coup classique qui coupe toutes les cordes et propulse les objets alentours. Cette adaptation résulte de l'observation des playtesteurs. En cherchant à couper le fil qui les reliaient à un objet, ils coupaient involontairement les fils alentours provoquant une frustration visible.

- Quand le joueur connecte deux objets, la corde s'ajuste automatiquement : le point d'attache passe du personnage au nouvel objet, et la longueur de la corde augmente de la distance entre le joueur et sa nouvelle cible. Cela permet d'éviter d'appliquer des forces involontaires aux objets reliés lors de cette action.
- La création de la corde et le choix de sa longueur se fait désormais en une action simple. Au lieu de relier puis de rallonger et raccourcir sa corde à une longueur voulue, le joueur peut la définir en maintenant enfoncée la touche de « relier » et en se déplaçant avec son personnage.

5.3 FONCTIONNEMENT INTERNE

Pour pouvoir personnaliser facilement la corde et son comportement, plusieurs outils ont été mis en place.

5.3.1 Interface de personnalisation

Un certain nombre de variables ont été exposées au Game-Designer afin de faciliter la personnalisation de la corde. Ces variables permettent de changer le rendu de la corde (épaisseur, longueur, material, etc.) et son comportement en jeu (friction, amortissement, force de tension, multiplicateur de gravité, etc.). Il est aussi possible de désactiver des parties de la simulation pour optimiser les performances. Par exemple, si on utilise une corde uniquement pour suspendre un objet hors de portée du joueur, on peut facilement désactiver les collisions de la corde.

5.3.2 Debug

Une fonction de debug peut aussi être activée pour voir la position des particules en temps réel et leur écartement. Cela permet aussi d'afficher un certain nombre d'informations supplémentaires sous la forme de texte (longueur actuelle de la corde, état, force appliquée aux objets reliés, etc.). Ce système permet de mettre en évidence les problèmes rencontrés lors de la réalisation du prototype.

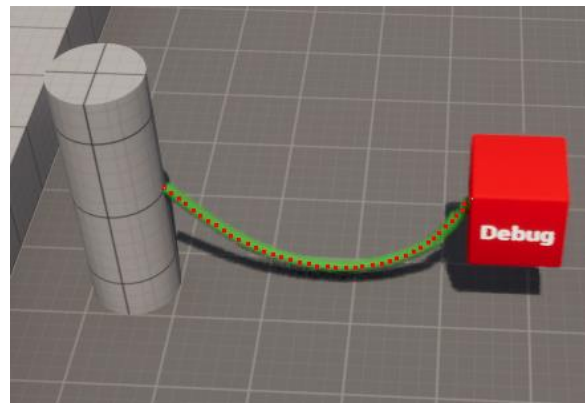


Figure 5-9 : Affichage des positions des particules en temps réel

5.4 OPTIMISATION

De nombreuses petites améliorations ont été faits pour permettre au joueur de créer un chaos raisonnablement grand dans sa partie sans lag notable. Ainsi, en début de frame, il a été implémenté une phase de filtration des particules susceptibles d'entrer en collision (Broadphase). Cette phase permet de réduire les calculs de résolution de collisions (Narrow Phase).

Les cordes déjà placées en jeu servent surtout à suspendre des objets et sont donc constamment tendues. Nous avons donc réduit drastiquement le nombre de particules de ces cordes et même empêcher le calcul de collisions pour certaines

Enfin l'implémentation de la technique de Small Steps (c.f. Substepping) de Macklin et al. (2019) a permis de gagner un nombre notable de frames par seconde tout en rendant la simulation plus réaliste.



Figure 5-10 : Scène chaotique tournant à moins de 20ms/frame

6 ANALYSE REFLEXIVE

6.1 DISCUSSION

6.1.1 Choix créatifs

Le choix de centrer notre jeu sur la simulation de cordes physiquement plausibles dans un environnement entièrement dynamique est un pari ambitieux et risqué. Cependant, la littérature a permis de mieux appréhender ces notions complexes et d'adapter les solutions existantes à notre situation et à nos attentes.

La démarche itérative utilisée tout au long de ce travail, par laquelle la recherche nourrit le design et vice-versa, semble avoir été appropriée à nos besoins. En effet, d'un côté nous avons besoin de retours réguliers des joueurs sur les choix de design, et, de l'autre, d'un temps d'assimilation et d'expérimentation de la littérature sur le sujet. Cela nous a permis d'axer le temps imparti sur les éléments essentiels au gameplay que nous imaginions, tout en expérimentant avec les éléments de recherche à notre disposition.

Après l'obtention d'une corde réaliste, nous avons choisi d'en styliser fortement son comportement, privilégiant l'ergonomie de son utilisation. Ce faisant, la corde conserve les caractéristiques essentielles d'un comportement physique crédible mais offre une maniabilité adaptée au contexte de son intégration dans le TFE. En effet, dans une simulation parfaitement réaliste, le poids des objets reliés limite considérablement les mouvements du personnage, à moins de les rendre extrêmement légers.

Par exemple, faire tourner des objets pour les propulser en coupant la corde ne serait plus possible dans une telle situation. Le personnage serait fortement impacté par la force de l'objet relié et serait comme planté dans le sol. Dès lors que la corde cesserait d'exercer une force de tension entre les deux corps, le personnage serait également propulsé (c.f. *Annexe 4 : Démonstration de la tension d'une corde réelle entre deux corps sur une surface sans friction*). C'est pourquoi nous avons sciemment diminué les forces des objets reliés appliquées sur le joueur. Les catégories de poids servent le même but en facilitant le choix de comportement des objets reliés. En effet, tous les objets d'une même catégorie de se comporteront sensiblement de la même manière.

6.1.2 Choix techniques

Le Cable Component natif d'Unreal Engine aurait pu être une solution de choix pour notre projet mais ses limites (difficulté de personnalisation de la solution, absence de transmission de forces aux objets reliés) sont rapidement apparues comme des arguments rédhibitoires à son utilisation (c.f. *Annexe 1 : Portfolio de R&D*). D'autres solutions (comme les chaînes de « Static Mesh Components » reliés entre eux par des « Physics Constraint Components »), ont également été rejetées dû à leur instabilité et/ou à leurs performances désastreuses.

L'utilisation de PBD (c.f. 3.2.3.2) de Müller et al. (2006) et de la solution de Small Steps (c.f. 3.4.1) de Macklin et al. (2019) nous ont permis d'obtenir une solution stable et suffisamment plausible (c.f. 0). De plus, cette solution laisse une grande marge de manœuvre au niveau de l'optimisation puisqu'il est très facile de réduire le nombre de substeps ou de particules en se basant sur l'état de la corde et du

joueur. Par exemple, il est possible de réduire ces variables quand la corde se stabilise ou quand le joueur est loin. Enfin, d'autres papiers de recherche sont venus compléter cette solution initiale en la rendant plus performante et plus stable (c.f. 6.4).

6.2 POINTS FORTS DU TFE

6.2.1 Originalité

Le plus gros atout de notre projet réside dans l'originalité de sa mécanique principale. Quelques jeux ont exploité les cordes dans leur gameplay mais aucun ne se rapproche entièrement de notre jeu :

- Dans Unravel 1 et 2, la corde est bien leur mécanique principale mais ce sont tous les deux des side-scrollers 2D.
- Des jeux comme Human Fall Flat et Party Animals ont aussi des puzzles basés sur des cordes mais ce ne sont que sur de courts passages.
- Webbing Journey est peut-être celui qui s'approche le plus de notre concept, mais les fils créés par le joueur n'ont aucune simulation de collision.
- Enfin, c'est le nouveau jeu de Windup Games (le studio à l'origine d'Unravel) s'inscrit un peu comme une suite des Unravel, du moins sur le plan du gameplay, mais dans un univers en 3D cette fois-ci. Cependant, la mécanique s'apparente bien plus à un grappin amélioré qu'au concept de relier des objets que nous présentons ici.

Ainsi le mix entre une corde pseudo réaliste, la capacité de tout relier et un environnement entièrement dynamique fait de notre jeu un concept unique à l'heure actuelle.

6.2.2 Plausibilité

L'un des piliers de notre approche consiste à transposer le fonctionnement physique d'une corde réelle dans notre jeu, puis d'en faciliter son utilisation et d'amplifier ses côtés ludiques. Comme dit plus haut (c.f. 6.1.1), nous avons fait le choix de styliser la simulation physique de la corde afin de la rendre à la fois naturel et amusante, à l'image de *Just Cause* et de son grappin aux capacités surnaturelles.

Dans un premier temps, utiliser l'algorithme de PBD (c.f. 3.2.3.2) de Müller et al. (2006) a permis de construire une base solide et stable garantissant des comportements prévisibles et cohérents. Cette base est essentielle pour l'immersion du joueur et pour faciliter l'approche de la mécanique en la faisant réagir comme on pourrait s'y attendre.

Dans un second temps, l'exagération des valeurs de simulation s'est avant tout basée sur les retours de playtests, le but étant de privilégier le Game Feel (c.f. *Game Feel*) de la mécanique au détriment du réalisme. Par exemple, tous les objets sont divisés en cinq catégories de poids (très léger, léger, moyen, lourd et très lourd). Par exemple, les objets très lourds ne peuvent quasiment pas être déplacés, les objets lourds ne ralentissent le joueur, et les trois autres catégories peuvent être aisément déplacés par le joueur avec plus ou moins de force.

6.2.3 Chaos

Notre corde, couplée à un environnement entièrement dynamique, crée un espace de jeu dans lequel les joueurs expérimentent une sensation de puissance créative et destructrice (d'après les joueurs, « *Ce jeu donne envie de tout casser* »). Parmi les playtesters, certains s'amusaient à enchevêtrer les éléments du décor dans des configurations improbables, d'autres à pousser les limites du jeu à l'extrême en reliant tous les objets de l'environnement, ou d'autres encore à casser tous les objets destructibles.

Cette dimension de *Sandbox* (jeu dans lequel la curiosité et la créativité du joueur sont fortement stimulées) émerge naturellement de la stabilité de la simulation de la corde (c.f. 0), de l'intuitivité de la mécanique (c.f. 5.2) et de nos choix créatifs (c.f. 6.1.1). Enfin, les feedbacks visuels et sonores de chaque impact, destruction d'élément ou ouverture de porte viennent sublimer le tout en gratifiant le joueur dans la création de ce chaos.

6.2.4 Game Feel

Grâce aux nombreuses améliorations faites sur les actions de manipulation de la corde (c.f. 5.2) ainsi que de la simulation de la corde en tant que telle, notre jeu a un Game Feel satisfaisant et gratifiant. Comme la mécanique est complexe, en faciliter son utilisation est primordiale pour ne pas rendre l'expérience pénible.

Les frustrations liées à l'utilisation de la corde ont rarement été formulées par les joueurs. C'est l'observation minutieuse de leur manière de jouer qui nous a permis de mettre le doigt sur les problèmes de notre implémentation. L'utilisation de la mécanique est maintenant fluide et les joueurs expérimentent de plus en plus avec la corde, qu'ils considèrent désormais comme un outil ou un jouet plutôt qu'une contrainte.

6.3 FAIBLESSES ET PISTES D'AMÉLIORATIONS

6.3.1 Performances

Bien que satisfaisantes au vu de l'état d'avancement du projet, on souhaite pouvoir permettre bien plus de liberté au joueur. En effet, le point névralgique de la solution actuelle est le nombre de collisions à calculer. Celui-ci est directement dépendant du nombre de particules dans le monde est donc essentiellement de la longueur des cordes. Ainsi le joueur peut très facilement poser plus d'une cinquantaine de petites cordes sans ressentir de problème de lag. Mais dès lors qu'il crée des cordes à leur longueur maximale, les performances chutent rapidement.

Certaines techniques ont déjà été mises en place pour améliorer ce phénomène (c.f. 4.5), et la technique de substepping (c.f. 3.4.1 de Macklin et al. (2019)) a aidé à stabiliser la simulation. En revanche, un certain nombre d'améliorations peuvent encore être mises en place.

6.3.1.1 Simulation des cordes hors-champ

La gestion des cordes non visibles à l'écran présente une complexité insoupçonnée. La solution naïve est de simplement désactiver la physique des cordes hors-champ. Or, une réactivation brutale de la simulation d'une corde peut générer des problèmes de snapping (objet ou particule qui se

téléporte à l'endroit où elle devrait être) si jamais les objets reliés étaient encore visibles et continuaient de bouger. Le problème s'étend alors à tous les objets interconnectés et nous devons prendre en considération l'ensemble du réseau.

Ainsi il n'est plus envisageable de calculer ces réseaux à chaque pas de temps. Une solution possible est de stocker ces réseaux et ne les actualiser que lorsqu'un de leurs acteurs devient visible ou invisible ou quand un acteur est rajouté ou enlevé de ce réseau.

6.3.1.2 Réduction adaptative de la fréquence d'actualisation des particules

A l'image du *sleep state* des moteurs de simulation physique modernes (Unreal Engine en faisant parti), chaque particule des cordes pourrait être désactivée en dessous d'un certain cap de vitesse. On pourrait même imaginer une réduction adaptative de la fréquence d'actualisation, et par extension du nombre de substeps par frame, des particules en fonction de leur vitesse.

6.3.1.3 Réduction adaptative du nombre de particules

Nos observations révèlent que les cordes posées présentent généralement un comportement physique stable, n'interagissant significativement qu'avec les objets qu'elles relient. De plus, une corde tendue qui n'est en contact avec aucun objet n'a aucun intérêt d'avoir une centaine de particules simulées.

Prenons l'exemple d'un objet se balançant suspendu à une poutre, la réduction drastique du nombre de particules n'aurait aucun effet visuel significatif sur la simulation de la corde, ou même son aspect. A l'image du *sleep state* (c.f. 6.3.1.2), on pourrait imaginer un état de désactivation des particules intermédiaires en fonction de son état (tendue, posée, etc.).

6.3.1.4 Parallélisation des calculs

Enfin, la mise en place d'un système de parallélisation (Multi-Threading) des calculs les plus intensifs est une piste d'amélioration prometteuse. En effet, les cordes n'interagissant pas directement entre elles, on pourrait imaginer effectuer une partie des calculs en simultanée, notamment la détection et la résolution des collisions.

6.3.2 Elasticité indésirée et manque d'interaction physique

En l'état, la corde présente une élasticité assez importante. Cette élasticité empêche parfois le joueur d'être précis et peut créer de la frustration. Par exemple, empiler des objets ou les placer à des endroits précis sont des actions difficiles à réaliser. Ce problème est dû à des approximations de transmission des forces du système au moteur physique d'Unreal Engine, responsable de la simulation des Rigid Bodies du jeu.

Une autre faiblesse résultante de cette séparation des systèmes est l'interaction de la corde avec les objets qu'elle touche. En effet, on s'attend à ce qu'une corde tendu applique des forces aux objets qui cherchent à la traverser. Or, aucune force ne leur est transmis et, s'ils poursuivent leur trajectoire, ils finissent par la traverser.

Ces problèmes résultent du peu de contrôle sur le moteur physique d'Unreal. Comme la solution de ce mémoire fonctionne indépendamment du moteur physique d'Unreal, il n'est pas possible d'actualiser les positions des objets à chaque substep. Pire, le changement de position des Rigid Body n'est pas considéré comme un changement physique et leur vélocité n'est donc pas adaptée. Les forces

de la corde sont donc transmises via la fonction « Add Force At Location ». Ces forces sont obtenues par la dérivation de la vitesse des points d'accroche qu'on multiplie par la masse de l'objet.

Enfin, bien que la simulation de la corde ne soit pas dépendante du framerate, les forces appliquées aux objets le sont.

C'est un problème épineux qui nécessite une grosse refonte du système actuel. La piste la plus probable est celle d'utiliser un moteur custom unifié. Ce moteur physique se substituerait donc entièrement au moteur natif d'Unreal et serait autant responsable de la simulation de la corde que celle des Rigid Bodies, comme le suggère les travaux de Bender et al. (2017) sur les systèmes hybrides.

6.4 PROLONGEMENTS

Les prolongements détaillés ci-dessous sont des pistes d'amélioration déjà en bonne voie de réalisation. Il est possible de voir l'état d'avancement actuel dans le projet contenu dans l'*Annexe 3 : Proof of Concept*. Le processus d'amélioration est déjà lancé afin de continuer le développement du jeu dans l'année à venir.

6.4.1 Moteur physique unifié

La mise en place d'un système physique unifié est une voie d'amélioration que nous souhaitons approfondir. Müller et al. (2020) proposent une adaptation du système de PBD de Müller et al. (2006) pour le faire également fonctionner avec des Rigid Bodies. Bien qu'ambitieuse, cette solution nous permettrait d'avoir un contrôle précis sur les éléments à optimiser et tout serait simulé selon l'approche des changements de positions (c.f. 3.2.3).

Actuellement, pour fonctionner correctement, nous avons sectionné une frame en substeps de même durée (c.f. 3.1.2.2). Pour ne plus avoir à se préoccuper de cette contrainte, Macklin et al. (2016) présentent une évolution majeure de PBD classique : XPBD (Extended Position-Based Dynamics). Leur utilisation d'un multiplicateur Lagrangien total leur permet, entre autres, de s'émanciper de ce pas de temps fixe. Cela faciliterait certaines optimisations discutées plus haut comme la réduction du nombre de substep (c.f. 6.3.1.2). Cela permettrait également d'introduire un aspect d'élasticité, cette fois-ci voulu et contrôlé, à notre solution.

6.4.2 Détection de collisions

S'émanciper du moteur physique d'Unreal Engine signifie implémenter notre propre solution de calcul de collisions. Cette solution est en cours de développement.

6.4.2.1 Broad Phase

La solution actuellement implémentée dans le prototype repose sur une approche naïve. On utilise les AABB des objets générés par Unreal et on itère sur l'ensemble des corps et particules du niveau pour détecter les paires qui se superposent. Cette approche bien que fonctionnelle, révèle rapidement ses limites dans des scènes chargées comme le premier niveau de notre jeu.

Pour pallier cette contrainte, nous envisageons d'utiliser la méthode de Sweep & Prune (Müller, 2025). Cette technique permet de classer les objets selon leur alignement sur les axes du monde. En triant de manière croissante selon leur position sur les axes puis en comparant uniquement les objets proches les uns des autres, on peut réduire drastiquement les temps de calculs de cette phase.

6.4.2.2 Narrow Phase

Les moteurs de simulation physique utilisent souvent les mêmes algorithmes pour la détection précise des collisions de la Narrow Phase, à savoir Gilbert-Johnson-Keerthi (GJK) et Expanding Polytope Algorithm (EPA). L'avantage de ces algorithmes est qu'ils sont capables de déterminer la collision entre n'importe quel objet convexe (sphère, cube, cylindre, hull). Cela s'inscrit ainsi dans la volonté de faire un système unifié comme indiqué plus haut.

Pour faire nos calculs dans le prototype, nous nous basons sur les colliders générés dans Unreal Engine, ce qui permet de toujours utiliser la puissance de ses outils de génération de colliders sans avoir à les recoder également.

6.4.2.2.1 GJK

L'algorithme GJK, expliqué par Winter (2020), permet de détecter efficacement si une collision a bien eu lieu entre deux objets convexes. Son principe est très simple et se base sur une différence de Minkowski, c'est-à-dire la soustraction de l'ensemble des sommets (vertices) de l'objet A avec celui de l'objet B. Si cette différence englobe l'origine, c'est qu'une collision a eu lieu. Cependant cet algorithme ne renvoie qu'un booléen et ne nous donne pas suffisamment d'informations pour résoudre la collision.

6.4.2.2.2 EPA

EPA, détaillé par (Winter, 2020b), est en quelque sorte une extension de GJK puisqu'il utilise certaines informations calculées dans le premier algorithme pour renvoyer la normale et la profondeur d'une collision entre deux objets. Pour ce faire, il « gonfle » progressivement un polyèdre autour de l'origine. Si un vertex est dans la direction de la normale de la face la plus proche de l'origine, il continue à gonfler le polyèdre jusqu'à ne plus en trouver. Une fois cela fait, la normale de la face la plus proche de l'origine correspond à la normale de la collision et sa distance à l'origine correspond à la profondeur de la collision. Grâce à ces informations, il est enfin possible résoudre les collisions.

6.4.2.2.3 Manifold

Enfin, pour stabiliser les contacts continus, comme un objet reposant sur le sol, on utilise ce qu'on appelle un manifold. Détaillé par Gregorius (2015), c'est un ensemble de 4 points de contact, générés par exemple par EPA, qui partagent la même normale de collision. Une telle structure permet de corriger la position d'un objet A par rapport à un objet B en le déplaçant depuis ces 4 points de contact, plutôt qu'un seul. Cette structure persiste aussi longtemps que les objets sont en contact stable. Dès lors que l'un de ces points n'est plus en contact, le manifold est supprimé et on retombe sur une résolution classique des collisions.

6.4.3 Système généralisé de contraintes

Enfin le troisième point important de l'évolution de notre système est la généralisation des contraintes. En effet, actuellement seules les contraintes de distance entre les particules d'une corde sont prises en compte et résolues. Cependant, pour rendre la corde encore plausible, il serait judicieux de prendre également en compte les contraintes angulaires d'une particule. En effet, une corde a une certaine rigidité qui n'est pas retranscrite dans notre solution.

De plus, dans la lignée de créer un système unifié, certains Rigid Bodies peuvent également être soumis à des contraintes. Dans notre jeu, on peut par exemple citer les portes coulissantes (contrainte de

distance maximale), les portes classiques et les trappes (contrainte angulaire), les horloges (contraintes de position), etc.

Dans cette optique, la conférence de Catto (2014) propose une vision unifiée des systèmes physiques comme un ensemble de contraintes à résoudre. Nous souhaiterions nous inspirer des idées de cette conférence pour généraliser la résolution des contraintes de notre jeu.

7 CONCLUSION

Notre TFE est un jeu « Physic-Based » dans lequel le joueur a la capacité de relier tout élément du monde à l'aide d'une corde simulée physiquement. C'est pourquoi nous nous sommes efforcés dans ce travail de comprendre et de développer des cordes dans un environnement 3D.

Grâce à l'implémentation du système de PBD de Müller et al. (2006), nous avons pu concevoir une corde physiquement plausible et stable. Nous avons ensuite réussi à optimiser la solution grâce aux Small steps de Macklin et al. (2019) pour la rendre raisonnablement fluide.

Il ressort de nos résultats que l'approche utilisée est une solution satisfaisante pour simuler de manière crédible et fluide les cordes dans un environnement 3D dynamique. Ainsi, la corde a pu être implémentée dans le projet de fin d'année. Grâce aux choix de stylisation de la physique tels que la réduction des forces appliquées sur le joueur et la catégorisation des poids des objets de l'environnement, nous avons rendu la corde intuitive et satisfaisante d'après les retours de playtests.

L'approche utilisée peut être améliorée en utilisant un système physique unifié (Müller et al., 2020). Cela permettra notamment de réduire drastiquement l'élasticité involontaire de la corde, de pouvoir améliorer les collisions de la corde et les forces qu'elle applique aux objets avec lesquels elle interagit.

8 BIBLIOGRAPHIE

- Bender, J., Müller, M., & Macklin, M. (2017). *A Survey on Position Based Dynamics*, 2017.
- Benson, H. (2015). *Mécanique* (5e éd). De Boeck.
- Catto, E. (Réalisateur). (2014). *Physics for Game Programmers : Understanding Constraints* [Enregistrement vidéo]. <https://www.youtube.com/watch?v=SHinxAhv1ZE>
- Cayère, C. (2024, septembre). *Simulation physique d'une corde dans un jeu vidéo*.
- Golding, J. (2014, avril 9). *Cable Component Plugin for Unreal Engine 4*. Unreal Engine. <https://www.unrealengine.com/fr/blog/cable-component-plugin-for-ue4>
- Gregorius, D. (2015). *Physics for Game Programmers : Robust Contact Creation for Physics Simulation*. GDC Vault. <https://gdcvault.com/play/1022193/Physics-for-Game-Programmers-Robust>
- Jakobsen, T. (2001). *Advanced Character Physics*. https://www.cs.cmu.edu/afs/cs/academic/class/15462-s13/www/lec_slides/Jakobsen.pdf
- Macklin, M., Müller, M., & Chentanez, N. (2016). XPBD : Position-based simulation of compliant constrained dynamics. *Proceedings of the 9th International Conference on Motion in Games*, 49-54. <https://doi.org/10.1145/2994258.2994272>
- Macklin, M., Storey, K., Lu, M., Terdiman, P., Chentanez, N., Jeschke, S., & Müller, M. (2019). Small steps in physics simulation. *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 1-7. <https://doi.org/10.1145/3309486.3340247>
- Müller, M. (Réalisateur). (2025, février 7). *Sweep and Prune Collision Detection with 10 lines of code* [Enregistrement vidéo]. <https://www.youtube.com/watch?v=MKeWXBgEGxQ>
- Müller, M., Heidelberger, B., Hennix, M., & Ratcliff, J. (2006). *Position Based Dynamics*. Virtual Reality Interactions and Physical Simulations (VRIPhys). <https://matthias-research.github.io/pages/publications/posBasedDyn.pdf>

- Müller, M., Macklin, M., Chentanez, N., Jeschke, S., & Kim, T. (2020). Detailed Rigid Body Simulation with Extended Position Based Dynamics. *Computer Graphics Forum*, 39(8), 101-112. <https://doi.org/10.1111/cgf.14105>
- Press, W., Teukolsky, S., Vetterling, W., & Flannery, B. (2007). *Numerical recipes : The art of scientific computing* (3. ed). Cambridge University Press.
- Roussel, J. (2014, juillet 2). *La méthode de Verlet*. FEMTO. <https://femto-physique.fr/analyse-numerique/methode-de-verlet.php>
- Roussel, J. (2015, juin 15). *La méthode d'Euler*. FEMTO. <https://femto-physique.fr/analyse-numerique/euler.php>
- Winter, I. (2020a, août 29). *GJK: Collision detection algorithm in 2D/3D*. Winter. <https://winter.dev/articles/gjk-algorithm>
- Winter, I. (2020b, novembre 17). *EPA: Collision response algorithm for 2D/3D*. Winter. <https://winter.dev/articles/epa-algorithm>

9 ANNEXES

9.1 ANNEXE 1 : PORTFOLIO DE R&D

Ci-joint de ce mémoire se trouve un dossier « Annexe1-PortfolioM1 » contenant le rendu du travail de l'année dernier. Dedans se trouve :

- Un fichier « FicherMemoire.pdf » : première version de ce mémoire réalisée en 2024.
- Un fichier « Portfolio.pdf » : portfolio de Recherche et Développement réalisé en 2024 et portant sur la simulation d'une corde dans un jeu vidéo fait avec Unreal Engine. Il est notamment fait une comparaison entre ma PoC et le Cable Component d'Unreal Engine.
- Un dossier « Prototype » contenant l'ensemble des fichiers du projet Unreal de ma première PoC.

9.2 ANNEXE 2 : DIAGRAMMES UML

9.2.1 Diagramme de cas d'utilisation

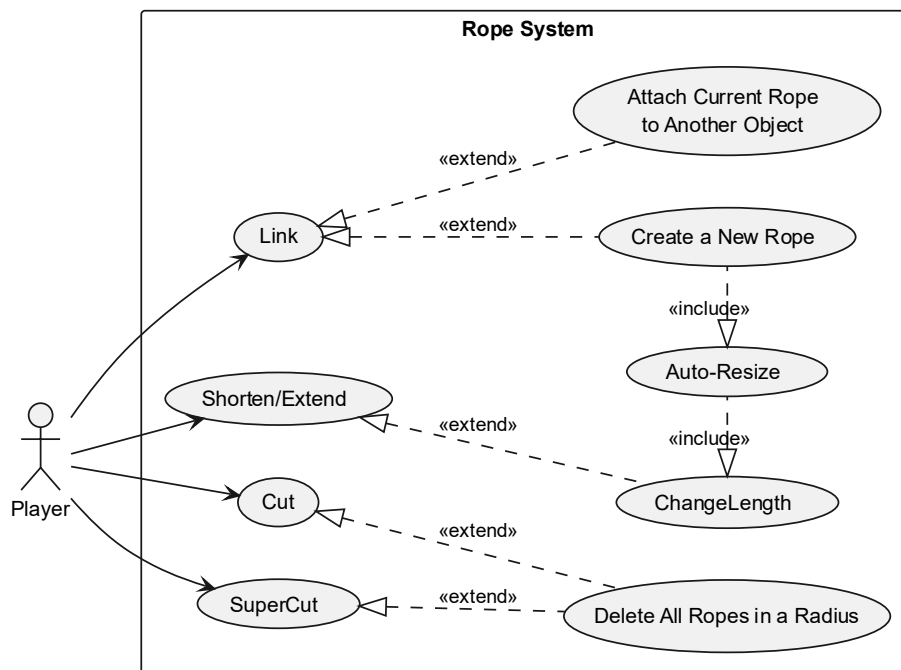


Figure 9-1 : Diagramme de cas d'utilisation du système de corde

Link : Pour pouvoir utiliser l'action de relier, le joueur doit être préalablement positionné près d'un objet fiable. Deux choses peuvent alors arriver lorsqu'il appuie sur la touche de *Link* :

- Si le joueur n'est pas déjà relié, une corde est créée, reliant l'objet visé au joueur. S'il reste appuyé sur la touche et se déplace simultanément, la longueur de la corde va s'adapter à la distance séparant le joueur de l'objet relié. La tension n'est pas prise en compte sauf si la corde atteint sa taille maximale.
- Sinon, le point d'accroche de l'extrémité de la corde associé au joueur va se transférer à l'objet visé.

Shorten/Extend : S'il est relié à un objet, en appuyant sur la touche de *Shorten* ou celle d'*Extend*, la corde va se redimensionner jusqu'à atteindre un minimum ou un maximum. La tension de la corde est prise en compte de sorte. Ainsi, lorsqu'elle est trop tendue, elle applique des forces sur l'objet relié et le joueur afin de les rapprocher.

Cut : Le joueur appuie sur la touche de *Cut*. S'il est déjà relié avec une corde, seule cette corde est supprimée. Sinon, toutes les cordes dans un petit rayon autour du joueur sont supprimées.

SuperCut : Le joueur appuie sur la touche de *SuperCut*. Il se met alors à charger et s'il relâche au bout d'un certain laps de temps, toutes les cordes dans un grand rayon autour du joueur sont supprimées.

Diagramme de classes.

9.2.2 Diagramme de classes

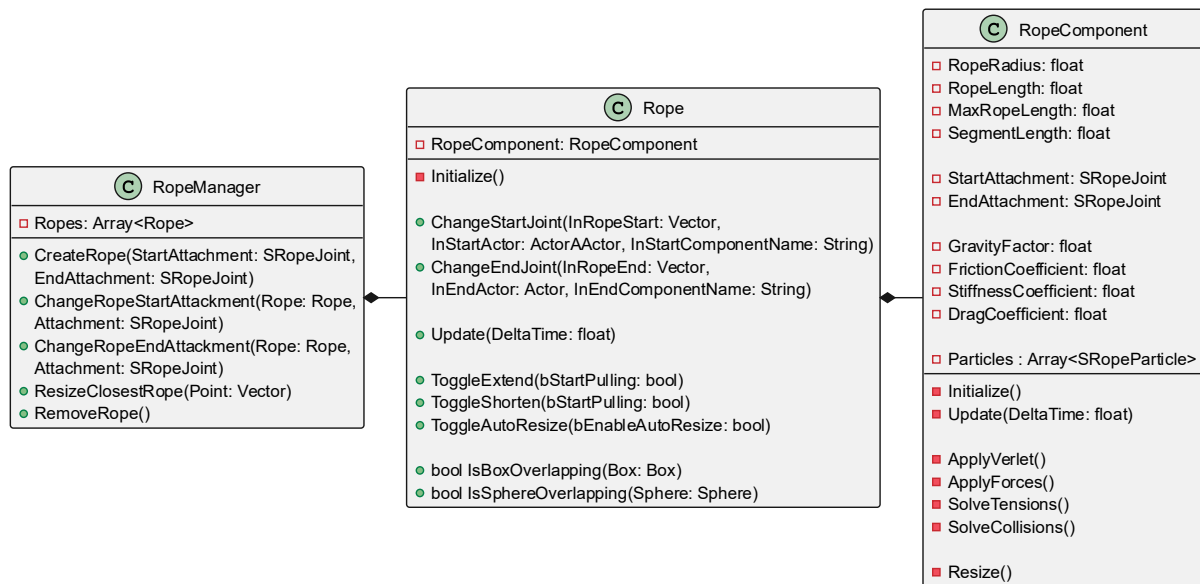


Figure 9-2 : Diagramme de classes du système de corde

9.2.3 Diagramme de séquence

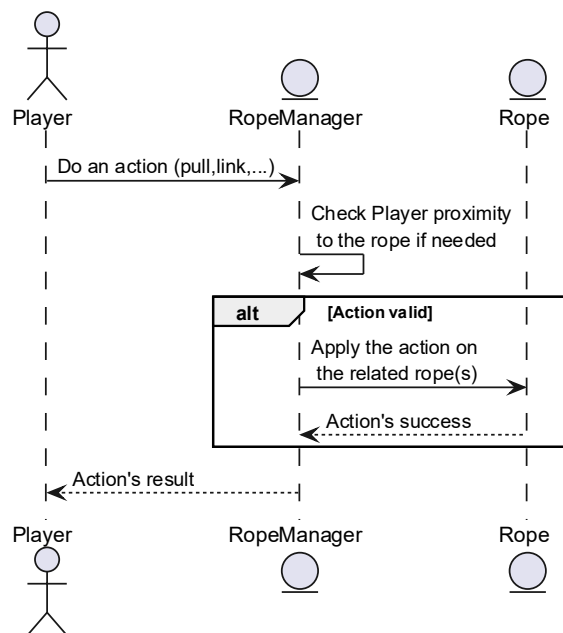


Figure 9-3 : Diagramme de séquence du dialogue entre le joueur et le système de corde

9.2.4 Diagramme d'activités

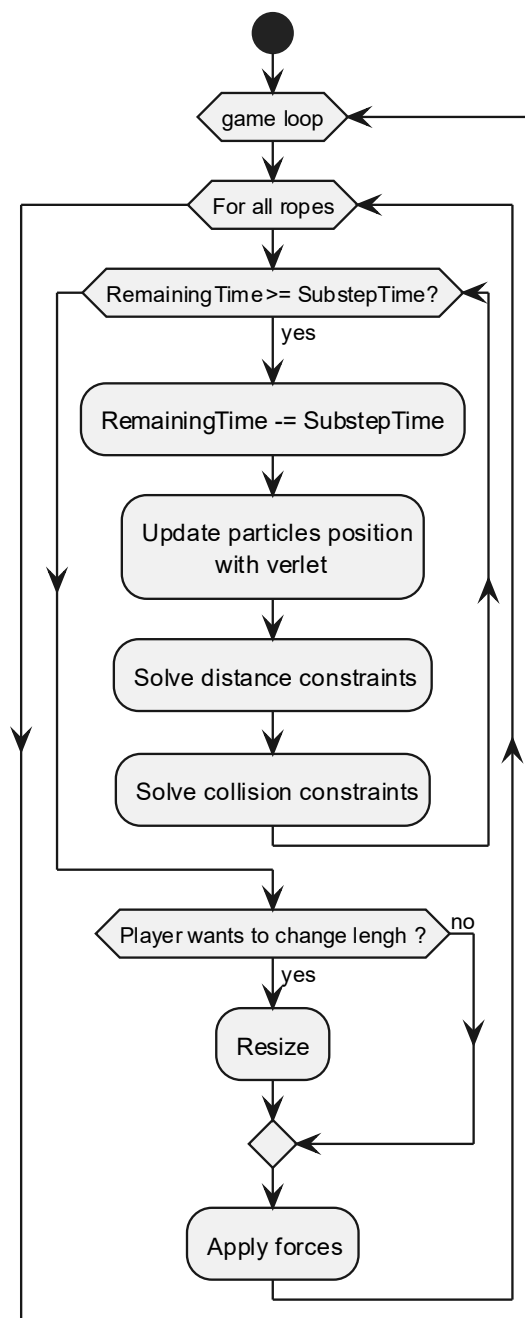


Figure 9-4 : Diagramme d'activités de la fonction Update du système de corde

9.3 ANNEXE 3 : PROOF OF CONCEPT

Ci-joint de ce mémoire se trouve un dossier « Annexe3-PoC » contenant la Proof of Concept de la corde. En lançant la solution depuis l'éditeur, il est possible de parcourir des pièces dans lesquelles les différents comportements de la corde sont mis en avant.

Enfin, la map « L_CustomPhysic » montre l'état d'avancement actuel du moteur physique custom.

9.4 ANNEXE 4 : DEMONSTRATION DE LA TENSION D'UNE CORDE REELLE

ENTRE DEUX CORPS SUR UNE SURFACE SANS FRICTION

Ci-joint de ce mémoire, se trouve un dossier « Annexe4-DemonstrationTensionCordeRelle ». A l'intérieur se trouve une vidéo montrant le comportement d'une corde reliant deux corps sur un lac de glace. On peut voir notamment les forces en jeu et la difficulté du corps tractant à faire bouger l'ensemble du système ou même à le contrôler.